

THE COMPONENTS OF A C PROGRAM

Outline

- The Main() function
- The #include Directive
- Variable Definition
- Function Prototype
- Program statements
 - ✓ Printf()
 - ✓ Scanf()
 - ✓ return
- Function Definition
- Program comments
- Using Braces
- Note of Accuracy

The Main() Function

- ✓ This is the only component that is required in every C program.
- ✓ In its simplest form, the main() function consists of the name main followed by a pair of empty parentheses (()) and a pair of braces ({}).
- ✓ Within the braces are statements that make up the main body of the program.
- ✓ Under normal circumstances, program execution starts at the first statement in main() and terminates at the last statement in main().
- ✓ **Lines 8 Through 23**

The #include Directive

- ✓ The #include directive instructs the C compiler to add the contents of an include file into your program during compilation.
- ✓ *An include file is a separate disk file that contains information needed by your*
- ✓ *program or the compiler.*
- ✓ Several of these files (sometimes called *header files*) *are supplied with your* compiler.
- ✓ You never need to modify the information in these files; that's why they're kept separate from your source code.
- ✓ Include files should all have an .H extension (for example, STDIO.H).
- ✓ You use the #include directive to instruct the compiler to add a specific include file to your program during compilation. **Line 2**
- ✓ **Assignment:** Explain the importance of Eight header files that are used in C programming. **(Send to my email by end of Wednesday 22nd Febraury 2023)**

Variable Definition

- ✓ *A variable is a name assigned to a data storage location.*
- ✓ *Any C program uses variables to store various kinds of data during program execution.*
- ✓ In C, a variable must be defined before it can be used.
- ✓ A variable definition informs the compiler of the variable's name and the type of data it is to hold. **Line 4**



The Function Prototype

- ✓ *A function prototype provides the C compiler with the name and arguments of the functions contained in the program.*
- ✓ It must appear before the function is used.
- ✓ *A function prototype is distinct from a **function definition**, which contains the actual statements that make up the function.*
- ✓ **Line 6**

Program statements

- ✓ The real work of a C program is done by its statements.
- ✓ C statements display information on-screen, read keyboard input, perform mathematical operations, call functions, read disk files, and carry out all the
- ✓ other operations that a program needs to perform.
- ✓ Remember that in your source code, C statements are generally written one per line and always end with a semicolon.
- ✓ Some of these statements include:

1.printf():

- The printf() statement (lines 11, 15, and 20) is a library function that displays information on-screen.
- The printf() statement can display a simple text message (as in lines 11 and 15) or a message and the value of one or more program variables (as in line 20).

2. **scanf():**

- The scanf() statement (lines 12 and 16) is another library function. It reads data from the keyboard and assigns that data to one or more program variables.
- The program statement on line 19 calls the function named product(). In other words, it executes the program statements contained in the function product().
- It also sends the arguments a and b to the function. After the statements in product() are completed, product() returns a value to the program.
- This value is stored in the variable named c.

3. **Return:**

- The return statement on line 28 is part of the function product(). It calculates the product of the variables x and y and returns the result to the program statement that called product().
- The return statement on line 22 returns a value of 0 to the operating system just before the program ends. This helps to inform the operating system that the program has ended.

The Function Definition

- ✓ A function is an independent, self-contained section of code that is written to perform a certain task.
- ✓ Every function has a name, and the code in each function is executed by including that function's name in a program statement. This is known as ***calling the function***.
- ✓ The function named `product()`, in lines 26 through 29, is a user-defined function.
- ✓ As the name implies, user-defined functions are written by the programmer during program development.
- ✓ This function is simple. All it does is multiply two values and return the answer to the program that called it.
- ✓ In a real C program, you probably wouldn't use a function for a task as simple as multiplying two numbers. We have done this here just to demonstrate how functions work.

Program comments

- ✓ Any part of your program that starts with `/*` and ends with `*/` or starts with `//` is called a *comment*.
- ✓ *The compiler ignores* all comments, so they have absolutely no effect on how a program works.
- ✓ You can put anything you want into a comment, and it won't modify the way your program operates.
- ✓ A comment can span part of a line, an entire line, or multiple lines.
- ✓ Many beginning programmers view program comments as unnecessary and a waste of time.
- ✓ This is a mistake! The operation of your program might be quite clear while you're writing it--particularly when you're writing simple programs.
- ✓ However, as your programs become larger and more complex, or when you need to modify a program you wrote six months ago, you'll find comments very useful. programming structures and

Using Braces

- ✓ You use braces ({}) to enclose the program lines that make up every C function-including the main() function.
- ✓ A group of one or more statements enclosed within braces is called a *block*.

Program Accuracy

- ✓ A computer is fast and accurate, but it also is completely literal.
- ✓ It doesn't know enough to correct your simplest mistake; it takes everything you enter exactly as you entered it, not as you meant it!
- ✓ This goes for your C source code as well. A simple typographical error in your program can cause the C compiler to choke and collapse.
- ✓ Fortunately, although the compiler isn't smart enough to correct your errors (and you'll make errors--everyone does!), it *is smart enough to recognize them as errors and* report them to you.